

CS-412 Software Security Lab 2:

Fuzzing libsixel with AFL++

Denis Fedosov
EPFL

Robin Nicole
EPFL

Antony Picard
EPFL

Kurz Tizian
EPFL

Abstract

We present a coverage-guided fuzzing campaign against `libsixel` [4], a C library that encodes and decodes SIXEL bitmap graphics. Using AFL++ [2] with AddressSanitizer, we fuzz multiple functions and compare results with a QEMU-mode black-box run on an uninstrumented binary. Our image-loading harness discovers a **bad-free** in `load_png` reducible to an 8-byte PoC, matching upstream issue #177. All harnesses, seed corpora, and analysis scripts are publicly available [3].

1 Q1 — Harness Design

1.1 Entry Points

We grepped `SIXELAPI`-tagged symbols in the installed header and kept only functions whose signatures accept raw buffers, file paths, or length parameters — the indicators of externally-controlled data. We selected three entry points. `sixel_decode_raw()` is the lowest-level decoder: every higher-level path (`sixel_decode`, `sixel_decoder_decode`, `sixel2png`) bottoms out here, making it the single point through which all attacker bytes flow. `sixel_decode()` was kept as a second decoder target since it exercises the callback-setup layer that `sixel_decode_raw` skips. `sixel_encoder_encode()` was chosen for the image-loading path: it dispatches to `libpng`, `libjpeg`, and the BMP loader, exposing far more branching than any pure decoder — and it was the only entry point that produced crashes. Rejected candidates: `sixel_encode`, `sixel_dither_initialize`, `sixel_helper_scale_image` (narrow arithmetic en-

try points, corpora saturate below 250 entries), and `sixel_decoder_decode` (I/O-indirection over the same core parser).

1.2 Data Flow and Guards

All harnesses share one skeleton: open the AFL++ input from `argv[1]`, read it into a heap buffer, validate, call the API, free all allocations. **Sixel-stream harnesses** (`harness_decode_raw`, `harness_decode`) pass the raw bytes directly to the library; output buffers allocated by `libsixel` (`pixels`, `palette`) are freed per iteration to stay compatible with persistent mode. **File-path harness** (`harness_load_image`) passes `argv[1]` to `sixel_encoder_encode`, which opens the file internally and dispatches to the appropriate loader; `stdout` is redirected to `/dev/null` so SIXEL escape sequences do not pollute timing.

Guard explanations. (a) `if (argc < 2) return 1` — protects against manual invocation during development, which would otherwise crash the harness itself and produce misleading reports. (b) `if (!f) return 1` — prevents NULL dereference in the subsequent `fseek` when AFL++’s initialisation probe fails to open a file. (c) `if (size == 0 || size > 1000000)` — an empty file gives no coverage signal; the 1 MB cap prevents raster-attribute headers like "100000;100000q" from triggering multi-gigabyte allocations that turn into timeouts/OOM rather than crash signals. (d) `if (!data) return 1` — a `malloc` failure under ASan’s shadow pressure would be saved as a false-positive crash; we exit cleanly. (e) `free(data); free(pixels)` — `sixel_decode_raw` transfers ownership of output buffers to the caller; fork mode would leak per-

process but the OS reclaims on exit, whereas persistent mode running thousands of iterations would OOM.

2 Q2 — Instrumentation and Sanitizers

The full build-flag table is in Appendix A (Table 1). In summary: **Build 1 (white-box)** uses `afl-clang-lto` for collision-free edge IDs plus `-fsanitize=address` to catch silent heap corruption, compiled static (`--disable-shared`) so AFL’s shm map reaches inside the library. **Build 2 (black-box)** is plain Clang with `-g -O1` and no sanitizer/AFL pass so it can run under `afl-fuzz -Q`. Both use `--disable-python` to work around a `make install` failure on Python 3.12+ (removed `imp` module).

Patches: none. The Dockerfile clones upstream, checks out tag `v1.8.6`, and builds with an unmodified `./configure && make` — no patch, `sed -i`, or `git apply`. Every crash is therefore a real bug against upstream, not a harness-side artefact.

3 Q3 — Seed Corpus and Dictionary

Seeds. Two hand-crafted SIXEL files (162B total) generated by `seeds/gen_seeds.py`. `epfl.six` (134B) renders the EPFL logo using two palette entries and exercises every major control code: run-length (!N), carriage-return (\$), newline (-). `minimal.six` (28B) is the smallest syntactically valid stream (ESC Pq...ESC \), used for harness sanity-checking.

Dictionary. Nine tokens in `sixel.dict` (Appendix B, Listing 1). Formally these are the *terminal symbols* of the SIXEL context-free grammar: the start symbol is a DCS string and the body is a sequence of colour, raster-attribute and sixel-band productions delimited by one-byte terminals. Discovering the DCS introducer ESC Pq by random mutation has probability $\approx 2^{-16}$ per byte pair; the dictionary

hands AFL++ these terminals directly so *havoc* can splice valid token boundaries from cycle 1.

Coverage contribution (from the 40 min `harness_decode_raw` status screen: `corpus_count=859`, `corpus_found=856`, `edges_found=469/1065`, `bitmap_cvg=44.04%`): *Havoc* produced 839/856 new entries (87 with new edges) — it dominates because dictionary tokens give havoc material to splice into plausible positions. *Deterministic* passes produced 16 entries (8 with new coverage), quickly exhausted on two small seeds. *Splice* contributed 0: it requires ≥ 2 corpus entries to combine, and with only two seeds at startup it never fired.

4 Q4 — Campaign Analysis

We focus on the image-loading campaign that found crashes (`harness_load_image`) and the QEMU black-box campaign (`harness_vanilla`); both ran 40 min on a single core. Status screens and edge plots: Appendix E.

harness_load_image (crashes found). Stability 100%. Corpus 1318 inputs (1314 found by mutation), the largest campaign since PNG/JPEG/BMP parsing exposes far more branches than the pure decoder; map density 12.66% (2114/16697), the large denominator including libpng, libjpeg and BMP, cycles done 0 (859 pending, 44 pending-favoured — corpus still actively growing); 2 saved crashes, 8 hangs. *Edges curve:* immediate sustained discovery — 977 edges at $t = 60$ s, 1065 at $t = 156$ s, 1221 at $t = 376$ s, 1384 at $t = 580$ s, 2114 at $t = 2400$ s, no plateau. Crashes appeared in the very first recorded interval (bug triggered within the first minute). **Did not saturate:** edge count still rising, 0 cycles completed, pending-favoured entries remained.

QEMU (harness_vanilla). Stability 100%; corpus 700 inputs (697 found); map density 0.71% (463/65536) — the low density is an artefact of QEMU’s fixed 65536-slot bitmap, not fewer edges: the raw count 463 matches the instrumented decoder’s 469, confirming QEMU explored the same code; cycles done 3 (21 entries pending); 0 saved crashes. *Edges curve:* 420 at $t = 60$ s, 463 by

$t = 307\text{ s}$ ($\approx 5\text{ min}$), flat for the remaining 35 min. **Saturated by minute 5:** three completed cycles, flat curve. Without ASan the fuzzer cannot distinguish silent memory corruption from normal exits, so it stops finding new paths once the structural variation space is exhausted.

5 Q5 — Crash Triage

The `harness_load_image` campaign produced **2 unique crashes** (`id:000000`, `id:000001`; signal 6/SIGABRT) plus 8 hangs; both reduce to the same bug (identical ASan trace). `harness_decode_raw` found none.

Reproduce. Running the harness on the crash yields an ASan **bad-free** (full trace in Appendix C, Listing 2). Allocation in `sixel_frame_new` (`frame.c:61`); bogus free at `loader.c:633`.

Minimize. `afl-tmin` reduced the 60-byte crash to 8 bytes (86.67%, 66 executions). The minimized PoC is exactly the PNG magic bytes (89 50 4e 47 0d 0a 1a 0a) with no IHDR — the shortest input that makes `libpng` fail and trigger the `longjmp` path leading to the bad free.

Root cause. `load_png()` declares `unsigned char **rows` at line 315 *without volatile* and initialises it to NULL at line 332 — *after* the first `setjmp` (line 323). When `libpng` detects a malformed PNG (here: missing IHDR) its error callback executes `longjmp(png_jmpbuf(png_ptr), 1)`, jumping back to the second `setjmp` at line 345. Per C11 §7.13.2.1, a non-volatile local written between `setjmp` and `longjmp` has *indeterminate* value after the jump; the compiler may keep `rows` in a register that `longjmp` restores to a stale value. Execution falls through to the `cleanup` label, which calls `sixel_allocator_free(allocator, rows)` on that garbage pointer.

Classification. ASan **bad-free**; CVE-2020-11721; CWE-825 (Expired Pointer Dereference — indeterminate pointer after `longjmp`) with secondary CWE-755; matches the root cause of `libsixel` issue #177 [1] (**heap-buffer-overflow** in the related PNG cleanup path, v1.8.6).

6 Q6 — Attack Surface Analysis

Real-world applications. `img2sixel` (shipped with `libsixel`) converts PNG/JPEG/BMP images to SIXEL for terminal display: any user passing an untrusted image triggers the full image-loading and decoding pipeline in their security context. `FFmpeg-SIXEL` streams multimedia (e.g. YouTube) directly to the terminal; because `FFmpeg` fetches and decodes remote media, an attacker controlling a media stream can deliver a crafted PNG that triggers the **bad-free** with no local file access required.

Attack scenario. The **bad-free** can lead to arbitrary code execution in an unsecured remote environment. An attacker crafts an image forcing `libsixel` to allocate memory in a predictable pattern; when the bad free fires, allocator metadata can be corrupted to write attacker-controlled data into an arbitrary location such as a GOT entry. The next indirect call redirects execution to shellcode. No authentication or interaction beyond image delivery is required.

Gaps in coverage. *Gap 1:* `sixel_encode()` (`src/tosixel.c`) — the raw encoder processes dimensions and colour-reduction parameters partially controlled by image metadata; integer overflow in dimension arithmetic could produce an undersized output buffer. *Gap 2:* `sixel_helper_normalize_pixelformat()` (`src/pixelformat.c`) performs arithmetic on width, height and bytes-per-pixel derived from a decoded image. Integer overflow here could produce an undersized destination, leading to an OOB write our harness never reaches since we do not call the normaliser after decoding.

7 Q7 — Binary-Only Fuzzing with QEMU Mode

The vanilla binary was built with plain Clang (no instrumentation, no ASan) against a separately compiled `libsixel-vanilla` and launched under QEMU. Both campaigns targeted `sixel_decode_raw` for 40 min. Side-by-side comparison in Appendix D

(Table 2).

Exec speed. QEMU 409 exec/s vs. instrumented 355 exec/s (15% QEMU advantage). Counterintuitive, but explained by workload: QEMU’s JIT adds first-encounter translation cost, but translated blocks are cached for every subsequent iteration of the short `sixel_decode_raw` function. ASan, by contrast, inserts a shadow-memory check before *every* load/store and runs heap quarantine on every alloc/free, dominating per-exec cost. Without ASan, compile-time instrumentation would be faster than QEMU; ASan inverts the ordering.

Edges. 469 (instrumented) vs. 463 (QEMU) — nearly identical. The gap reflects probabilistic hash collisions in QEMU’s fixed 65 536-slot bitmap: two distinct edges hashing to the same slot are counted once. The instrumented build assigns a unique compile-time ID to every CFG edge. Map-density figures (44% vs. 0.71%) are not comparable: the instrumented denominator is the real edge count (1 065) while QEMU’s is its fixed map size.

Corpus. 859 (instrumented) vs. 700 (QEMU). Collision-free edge IDs give a more precise novelty signal: two inputs differing only in which of two colliding edges they hit are distinct in the instrumented build but identical to QEMU, so the instrumented fuzzer saves more queue entries and explores broader mutations.

8 Q8 — Instrumentation Depth and Performance

Edge counts (from `total_edges` at startup): **harness_decode_raw 1 065 edges** — only what is reachable from `sixel_decode_raw` after linker dead-code elimination (encoder, scaler, pixel-format routines stripped). **harness_decode 1 057** — nearly identical, confirming both exercise the same decoder call graph. **harness_load_image 16 697** — much larger because `sixel_encoder_encode` pulls in libpng, libjpeg, the colour-reduction path and I/O wrappers. The 1 065 vs. 16 697 gap shows how entry-point selection determines instrumentation scope: the same library produces very different coverage

maps depending on which subsystem the harness calls.

Map density vs. total edges. Density is $d = e_{\text{reached}}/e_{\text{total}}$. Not all instrumented edges are reachable from `sixel_decode_raw`: encoder paths, pixel-format converters and write-side I/O are compiled in (with `--whole-archive`) but never called. Even among reachable edges, paths gated on rare valid sequences may need many mutations to discover within 40 min.

Three configurations (Appendix D, Table 3): no-sanitizer fork 542 exec/s, ASan fork 355, ASan persistent 23 717. *No-san* → *ASan fork* ($\approx 35\%$ slow-down): ASan inserts a shadow-memory check before every load/store and maintains a 1/8-address-space shadow map; per-op overhead plus higher memory footprint make each fork more expensive. *Fork* → *persistent* ($\approx 67\times$ speedup): fork mode pays `fork(2)` and CoW page-table copy on every case; persistent mode loops inside `__AFL_LOOP(1000)`, amortising fork cost over 1 000 iterations.

A Build Configurations

Flag	B	Effect / if omitted
<code>CC=afl-clang-lto</code>	1	AFL++ LTO: collision-free edge IDs. Without it, no coverage feedback.
<code>CC=clang</code>	2	Plain Clang. Required for <code>-Q</code> ; compiler instrumentation conflicts with QEMU.
<code>-fsanitize=address</code>	1	ASan red-zones, shadow memory, quarantine (CFLAGS+LDLFLAGS). Without: OOB/UAF silent.
<code>-g</code>	1,2	DWARF info; else ASan shows raw addresses.
<code>-O1</code>	1,2	Light opt; <code>-O0</code> bloats map, <code>-O2/O3</code> inlines too aggressively.
<code>--disable-shared</code>	1	Static lib. A <code>.so</code> bypasses AFL’s shm map (0% coverage).
<code>--disable-python</code>	1,2	Avoids <code>make install</code> failure on Python 3.12+ (removed imp).

Table 1: Build flags. B1 = instrumented (AFL++ + ASan); B2 = vanilla (QEMU).

B SIXEL Dictionary

Listing 1: Contents of `sixel.dict`.

```
header_pq="\x1bPq"    # DCS + 'q'
header_p="\x1bP"      # DCS no mode byte
terminator="\x1b\\"    # String Terminator
color_intro="#"       # palette / color
raster_attr="\\"      # raster attr intro
rle_intro="!"         # run-length intro
graphics_cr="$"       # graphics CR
graphics_nl="-"       # graphics newline
param_sep=";"         # param separator
```

Configuration	exec/s
No sanitizer, fork mode	542
ASan, fork mode	355
ASan, persistent mode	23 717

Table 3: Throughput of `harness_decode_raw`.

C Crash Triage — ASan Trace and Minimisation

Listing 2: Crash reproduction (abridged) and `afl-tmin`.

```
$ ./harness_load_image <crash>

ERROR: AddressSanitizer: attempting free on
address which was not malloc()-ed:
#0 free (harness_load_image)
#1 load_png loader.c:633
#2 load_with_built_in loader.c:889
#3 sixel_helper_load.. loader.c:1418
#4 sixel_encoder_encode encoder.c:1743
#5 main harness_load_image.c:16
Alloc: malloc <- sixel_frame_new frame.c:61
      <- load_with_built_in loader.c:885
SUMMARY: bad-free in free

$ afl-tmin -i <crash> -o out.png \
  -- ./harness_load_image @@
# 86.67% reduction (60 -> 8 bytes)
```

```
[AFL++ d4bd8d115f08] /lab # cat findings/load-image/default/fuzzer_stats
start_time      : 1778602812
last_update     : 1778605212
run_time        : 2400
fuzzer_pid      : 109
cycles_done     : 0
cycles_wo_finds : 0
time_wo_finds   : 90
fuzz_time       : 2341
calibration_time : 30
cmplog_time     : 0
sync_time       : 0
train_time      : 28
execs_done      : 810921
execs_per_sec   : 337.88
execs_ps_last_min : 376.75
corpus_count    : 1318
corpus_favored  : 191
corpus_found    : 1314
corpus_imported : 0
corpus_variable : 0
max_depth       : 14
cur_item        : 1273
pending_favs    : 44
pending_total   : 859
stability       : 100.00%
bitmap_cov      : 12.66%
saved_crashes   : 2
saved_hangs     : 8
total_timeout   : 5168
last_find       : 1778605212
last_crash      : 1778602870
last_hang       : 1778603747
execs_since_crash : 797199
exec_timeout    : 20
slowest_exec_ms : 0
peak_rss_mb     : 0
cpu_affinity     : 11
edges_found     : 2114
total_edges     : 16697
var_byte_count  : 0
havoc_expansion : 0
auto_dict_entries : 0
testcache_size  : 92381
testcache_count : 1318
testcache_evict : 0
afl_banner      : ./harness_load_image
afl_version      : ++4.41a
target_mode      : shmem_testcase default
command_line     : afl-fuzz -V 2400 -i seeds_images -o findings/load-image -- ./harness_load_image @@
```

Figure 1: AFL++ status screen — instrumented `load_image` campaign.

D Comparison and Performance Tables

Metric	Instr.	QEMU
Duration	40 min	40 min
Exec speed (exec/s)	355	409
Edges discovered	469	463
Total edge map size	1 065	65 536
Map density	44.04%	0.71%
Corpus count	859	700
Cycles done	2	3
Saved crashes	0	0

Table 2: Instrumented vs. QEMU on `sixel_decode_raw`.

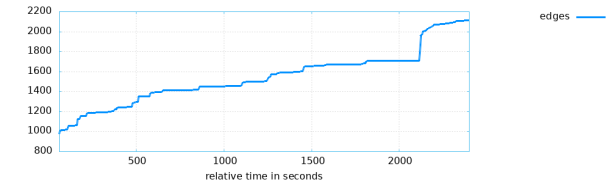


Figure 2: `afl-plot` edges graph — instrumented `load_image` campaign.

```

[AFL++ d4bd8d15f08] /lab # cat findings/qemu/default/fuzzer_stats
start_time      : 1778682612
last_update     : 1778685212
run_time        : 2400
fuzzer_pid      : 115
cycles_done     : 3
cycles_wo_finds : 0
time_wo_finds   : 94
fuzz_time       : 1694
calibration_time : 12
cmplog_time     : 0
sync_time       : 0
trin_time       : 693
execs_done      : 981295
execs_per_sec   : 408.87
execs_ps_last_min : 412.93
corpus_count    : 700
corpus_favored  : 41
corpus_found    : 697
corpus_imported : 0
corpus_variable : 0
max_depth       : 16
cur_item        : 544
pending_favs    : 0
pending_total   : 21
stability        : 100.00%
bitmap_cvg      : 9.71%
saved_crashes   : 0
saved_hangs     : 0
total_tmout     : 47
last_find       : 1778685200
last_crash      : 0
last_hang       : 0
execs_since_crash : 981295
exec_timeout    : 20
slowest_exec_ms : 0
peak_rss_mb     : 0
cpu_affinity     : 8
edges_found     : 463
total_edges     : 65536
var_byte_count  : 0
havoc_expansion : 0
auto_dict_entries : 0
testcache_size  : 1616885
testcache_count : 700
testcache_evict : 0
afl_banner       : ./harness_vanilla
afl_version      : ++4.41a
target_mode      : qemu shmem_testcase
command_line     : afl-fuzz -V 2400 -Q -i seeds -o findings/qemu -x sixel.dict -- ./harness_vanilla @@

```

Figure 3: AFL++ status screen — QEMU-mode campaign.

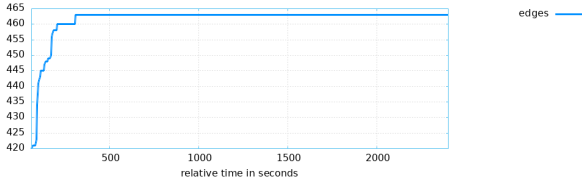


Figure 4: afl-plot edges graph — QEMU-mode campaign.

References

- [1] libsixel GH issue #177, 2023. <https://github.com/saitoha/libsixel/issues/177>.
- [2] AFL++ contributors. AFL++: a superior fork of AFL, 2023. <https://github.com/AFLplusplus/AFLplusplus>.
- [3] Denis Fedosov, Robin Nicole, Antony Picard, and Tizian Kurz. Cs-412 software security lab 2 — fuzzing campaign source, 2026. <https://github.com/github-fed/softsec-lab2>.
- [4] Hayaki Saito. libsixel — SIXEL encoder/decoder library, 2014. <https://github.com/saitoha/libsixel>.